Return values and local state

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

One connected return-and-local-state record.

Explain value flow, trace repeated calls, repair missing return and state the data assumptions.

Keep one habit: Keep local names, caller bindings, returned values, printed output and first error distinct.

MAIN LESSON · SECTION 1

Recall · Retrieve before notes

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Try the recall before opening notes. Then use a different colour or a labelled second line to correct your explanation.

- Without opening the old lesson, explain one key idea from Functions and arguments. Give an example and a mistake that the idea helps you avoid.

- Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

- Without opening the old lesson, explain one key idea from Expressions and operator order. Give an example and a mistake that the idea helps you avoid.

Functions and arguments

> **My remembered idea, example and correction**

Loops and trace tables

> **My remembered idea, example and correction**

Expressions and operator order

> **My remembered idea, example and correction**

W1 · Explain the procedure

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Main remaining(12,4)/(10,7)returns 8/3; caller sums 11. W2 remaining(8,3)/(8,8)/(8,3)stores a 5,b 0,c 5; print(a,b,c) displays 5 0 5. Pictured extra(2,5)/(4,1)returns7/5, caller prints 7 5. Local spaces/result not caller names in fresh script. Faulty print-only function returns None, not the visible number or 0; addition to None fails. Supplied valid integer assumptions remain separate from function semantics.

- Reference for this exercise: open this week's student lesson, "Return values and local state", at "See how it works" and "Follow a worked example". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

- Using the correct worked definition, execute result = remaining(15, 6), then doubled = result * 2. State both stored values and what the script displays before a print statement is added.

> **My reasoning and evidence**

MAIN LESSON · SECTION 6

W2 · Predict, trace and observe

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Main remaining(12,4)/(10,7)returns 8/3; caller sums 11. W2 remaining(8,3)/(8,8)/(8,3)stores a 5,b 0,c 5; print(a,b,c) displays 5 0 5. Pictured extra(2,5)/(4,1)returns7/5, caller prints 7 5. Local spaces/result not caller names in fresh script. Faulty print-only function returns None, not the visible number or 0; addition to None fails. Supplied valid integer assumptions remain separate from function semantics.

- Reference for this exercise: open this week's student lesson, "Return values and local state", at "See how it works" and "Follow a worked example". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

- Using the correct definition, store a = remaining(8, 3), b = remaining(8, 8), c = remaining(8, 3). State a, b and c. In a fresh script, explain why print(spaces) outside the function fails.

> **My reasoning and evidence**

MAIN LESSON · SECTION 7

W3 · Find the first error

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Main remaining(12,4)/(10,7)returns 8/3; caller sums 11. W2 remaining(8,3)/(8,8)/(8,3)stores a 5,b 0,c 5; print(a,b,c) displays 5 0 5. Pictured extra(2,5)/(4,1)returns7/5, caller prints 7 5. Local spaces/result not caller names in fresh script. Faulty print-only function returns None, not the visible number or 0; addition to None fails. Supplied valid integer assumptions remain separate from function semantics.

- Reference for this exercise: open this week's student lesson, "Return values and local state", at "Find and repair a mistake". Use the original worked values or code named in the question; the later pictorial case is a separate example. Keep this reference beside your workbook while practising.

- Use the faulty printing version with answer = remaining(11, 2). State the displayed number and the value stored in answer. Explain why print(answer + 1) fails, then repair the function and predict the repaired **output**.

> **My reasoning and evidence**

MAIN LESSON · SECTION 9

W5 · State an honest limit

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Main remaining(12,4)/(10,7)returns 8/3; caller sums 11. W2 remaining(8,3)/(8,8)/(8,3)stores a 5,b 0,c 5; print(a,b,c) displays 5 0 5. Pictured extra(2,5)/(4,1)returns7/5, caller prints 7 5. Local spaces/result not caller names in fresh script. Faulty print-only function returns None, not the visible number or 0; addition to None fails. Supplied valid integer assumptions remain separate from function semantics.

- A function body has spaces = capacity - used, then return spaces, then print("done"), all at the same indentation. What does answer = remaining(4, 4) store and display? Explain whether the final print can run.

> **My reasoning and evidence**

MAIN LESSON · SECTION 4

Try alone · Selected independent task

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Trace both calls and the final expression. If return total is removed from the function, does this final print complete? Would a separate print(total) outside the function have access to its **local variable**?

- Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Independent case: Use returns and keep local state local

```
def twice(n):
    total = 2 * n
    return total
first = twice(3)
second = twice(5)
print(first, second, first + second)
```

> **My independent explanation, trace and limits**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |

MAIN LESSON · SECTION 10

Project · Keep code and evidence

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Add a labelled **return value**s and local state **evidence** sheet to Python readiness portfolio. Show how you can use a returned value in a later calculation.

  Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

My project change, supporting evidence and remaining limit

MAIN LESSON · SECTION 11

Feedback · Repair and return

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Revisit this idea with your teacher after a delay. Familiarity is different from explaining and using it on a changed case.

Evidence	My record
Worked alone / hint / model	
First step I repaired and why	
New case checked later; date and result	
What I can now explain without notes	
One remaining question	

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Supported W2 record**

REFERENCE · SECTION 2

Reference · Supported W2 record

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use W2: Trace separate calls for **Return value**s and local state. Record the answer once; the lesson and workbook refer to the same attempt.

Given inputs	Procedure I chose	Middle steps	Result and check

> **The step I completed without help, and the hint I needed**

▼ Extra practice · Individual reasoning check

REFERENCE · SECTION 3

Extra practice · Individual reasoning check

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Use the worked case for **Return value**s and local state. Proposed explanation: “Printing a value returns that value”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

> **My explanation before partner discussion**

▼ Additional practice · Earlier W4 transfer

REFERENCE · SECTION 8

Additional practice · Earlier W4 transfer

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Each tray holds 5 counters. Define stock(trays, loose) to return the total using a **local variable** total. Assume nonnegative integer arguments. Store results for (2, 3), (0, 3) and (2, 0).

  Print the sum of the first two stored results. Predict that **output** and all three stored values. Call (2, 3) again to check fresh state.

> **My reasoning and evidence**

▼ **Extra practice · Pictured case**

REFERENCE · SECTION 12

Extra practice · Pictured case

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- **Return value**s survive their calls. A function returns a local calculation that the **caller** stores.

- Python 3: def extra(a,b): result=a+b; return result. Then first=extra(2,5); second=extra(4,1); print(first,second).

- Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

- Support used: alone / hint / worked example. This record is practice **evidence**.
- Project connection: For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation.

  Use actual Python 3 results to check the **prediction** after reasoning first.



Follow the labelled picture with the matching explanation. Open the vector diagram

My picture-based explanation and changed-case prediction

▼ **Reference · Project portfolio milestone**

REFERENCE · SECTION 13

Reference · Project portfolio milestone

> Keep the stated task, its own code/values and support record together.

Task and evidence record

- Project: Python readiness portfolio
- Current milestone: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a function on changed **input**s.
- For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Evidence to keep	My record
This week's input and deciding step	
Prediction and actual result (or labelled paper trace)	
One change since the previous checkpoint and why	

Evidence to keep	My record
One error or limit and the next test	

Explain what this week adds to your project. Reuse last week's record rather than starting again.

▼ **Extra practice · Predict, check and repair**

REFERENCE · SECTION 14

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- **Return value**s survive their calls. A function returns a local calculation that the **caller** stores.
- Which labelled step matches this detail? The final print displays 7 5; the second call does not overwrite first.
- 1. Read **caller** values
- 2. Trace first call
- 3. Trace second call

- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> ---
>
> ---
>
> ---
>
> ---

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Read **caller** values The final print displays 7 5; the second call does not overwrite first. Returning passes a value back to the **caller**.

Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Changed-case question and hint

- Changed-case practice: Add print(result) after the calls in a fresh script with no global result. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. NameError occurs because that local name is not defined in the **caller**.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: NameError occurs because that local name is not defined in the **caller**. A returned value and the function's internal name are different things.

Repair and keep the record

Review Functions and arguments (python-bridge/8). Goal: Distinguish a function definition from a function call. Short practice: Match every argument to a parameter for the two calls.

Why do **output**s differ, and what prints when both calls are removed?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

Review Expressions and operator order (python-bridge/3). Goal: Evaluate numeric expressions using Python's grouping rules. Short practice: Trace the operation order in both expressions.

Identify which picture step changes with parentheses and predict both results when the multiplier becomes four.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

> **What I repaired, the evidence I kept and the next test**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |
> | | |

▼ **Earlier wording, alternatives and picture transcripts**

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

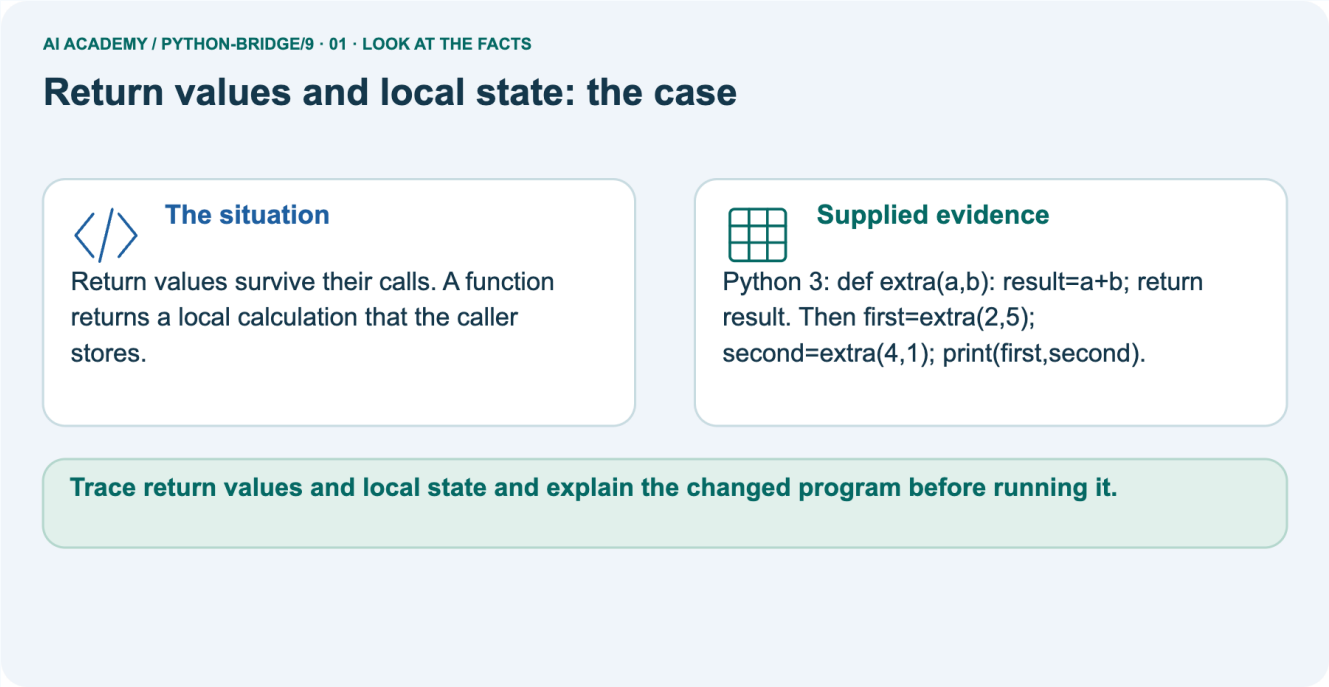
▼ **Reference · Six-picture response sheets**

REFERENCE · SECTION 15

Reference · Six-picture response sheets

> Use the picture relevant to a current gap.

01. Return values and local state: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

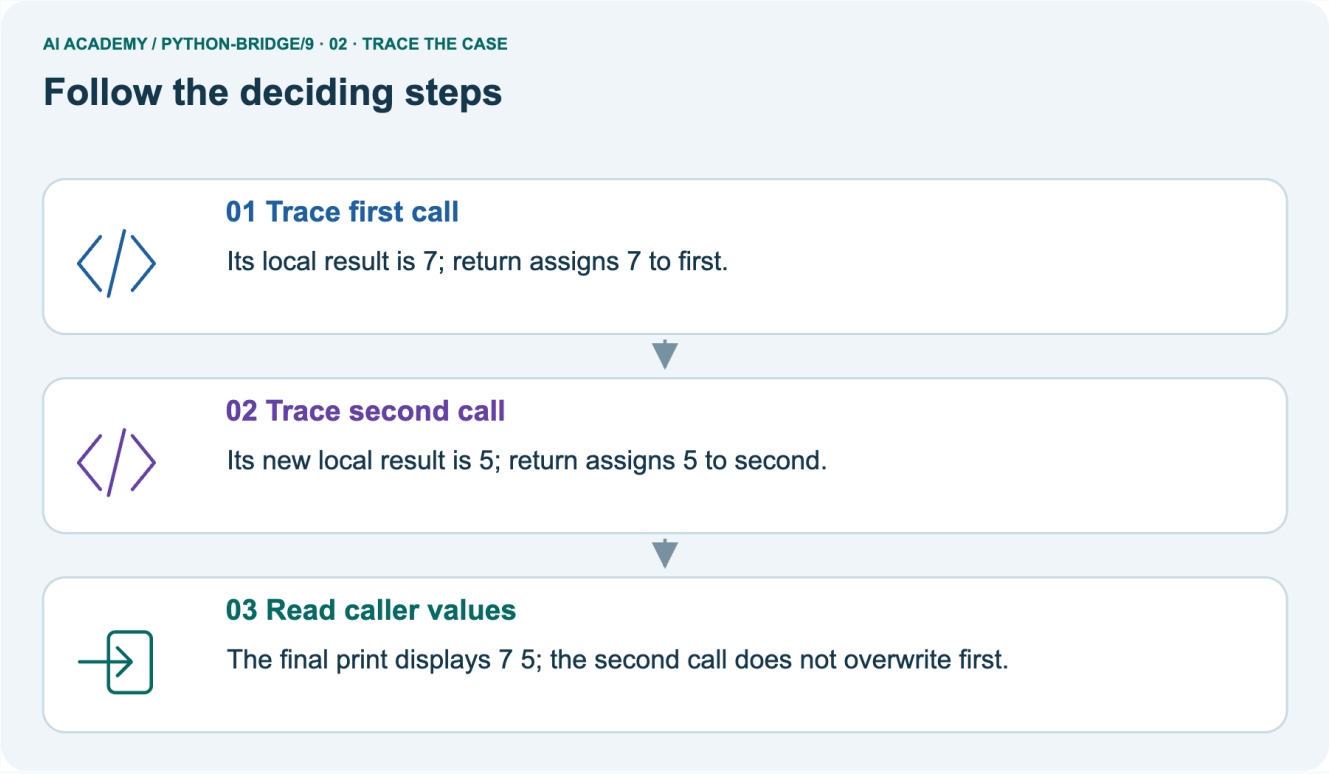
Return values survive their calls. A function returns a local calculation that the **caller** stores.
Supplied facts: Python 3: def extra(a,b): result=a+b; return result.

Then first=extra(2,5); second=extra(4,1); print(first,second).

Point to the given fact that matters most. Explain why it matters.

My picture explanation, deciding evidence and one limit

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Trace first call: Its local result is 7; return assigns 7 to first. → Trace second call: Its new local result is 5; return assigns 5 to second. → Read **caller** values: The final print displays 7 5; the second call does not overwrite first.

Point to each numbered stage and explain which recorded **input** it uses.

My picture explanation, deciding evidence and one limit

03. Two calls return two values

AI ACADEMY / PYTHON-BRIDGE/9 · 03 · INSPECT THE LESSON MODEL

Two calls return two values

Call	Inside this call	Back in the caller
remaining(12, 4)	spaces becomes 8	first receives 8
remaining(10, 7)	spaces becomes 3	second receives 3

The caller adds the returned values. Only print(first + second) displays 11.

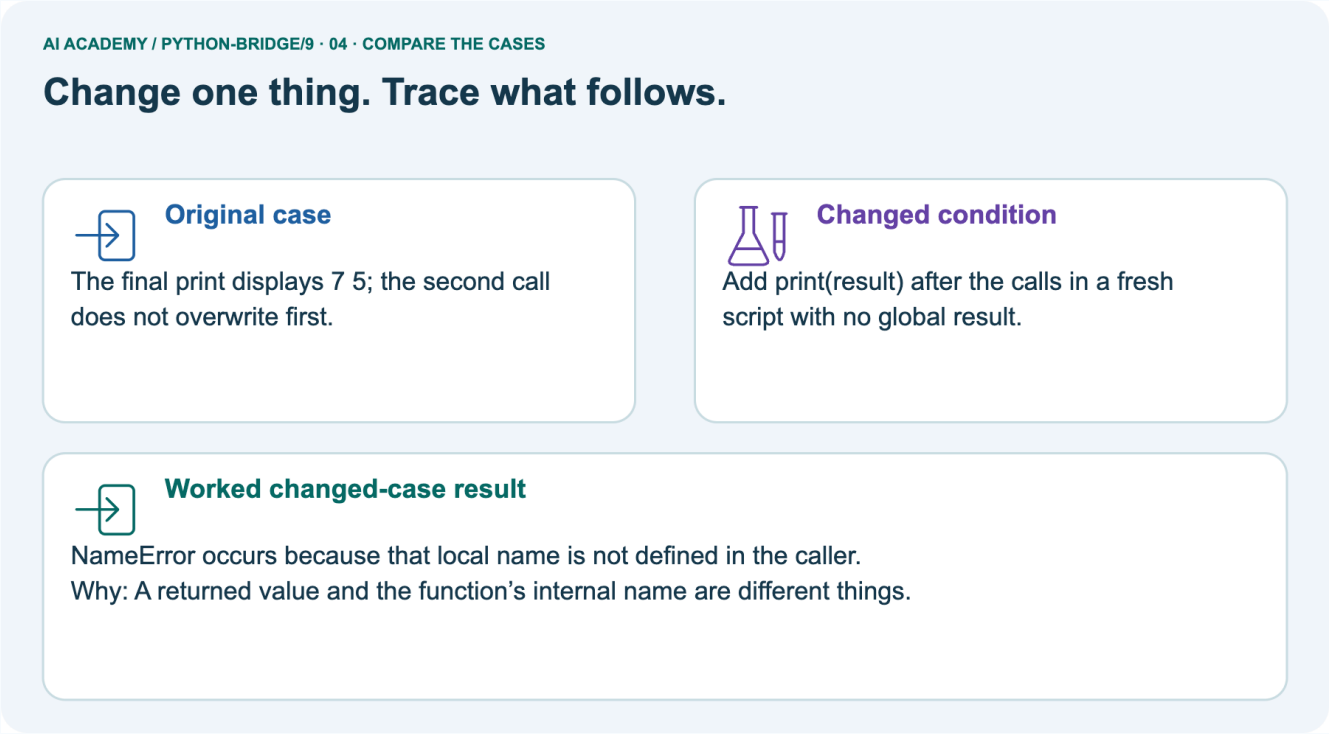
Follow the labelled picture with the matching explanation. Open the vector diagram

Two calls return two values. remaining(12, 4) | spaces becomes 8 | first receives 8. remaining(10, 7) | spaces becomes 3 | second receives 3

Read the labels and values. Explain the relationship the model shows.

My picture explanation, deciding evidence and one limit

04. Change one thing. Trace what follows.



Follow the labelled picture with the matching explanation. Open the vector diagram

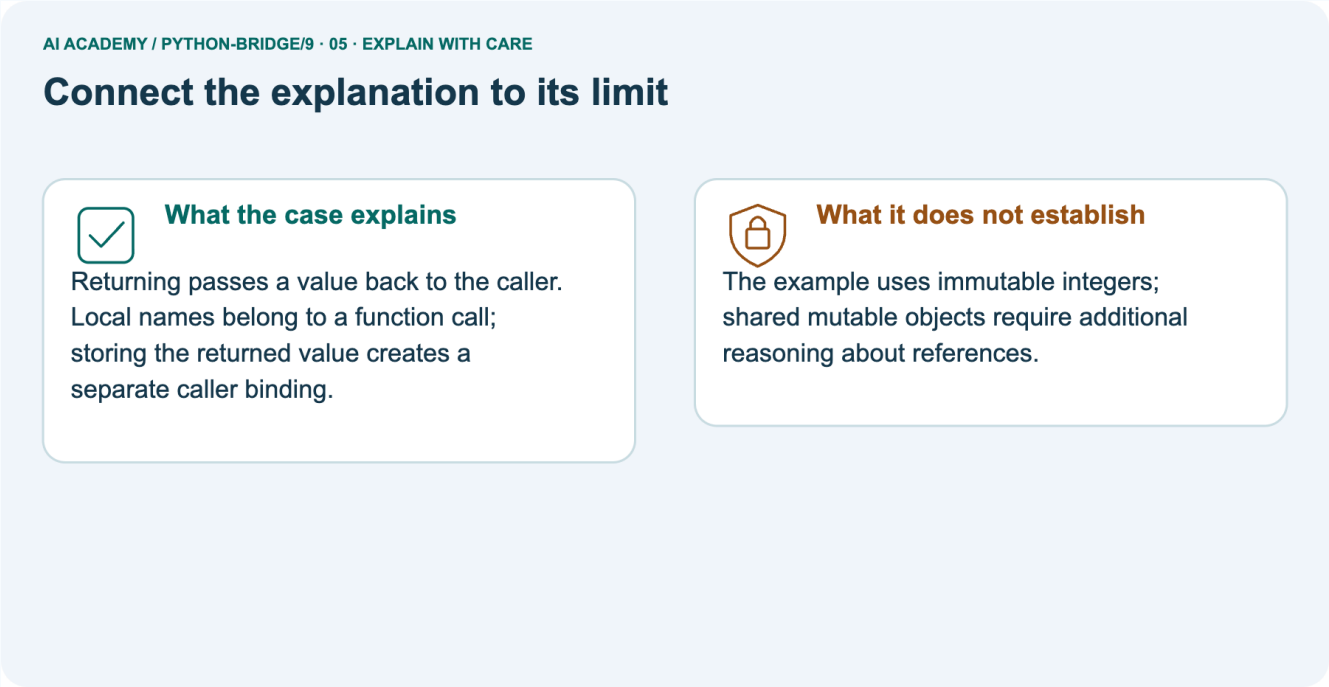
Original case: The final print displays 7 5; the second call does not overwrite first. Changed condition: Add print(result) after the calls in a fresh script with no global result.

Expected result: NameError occurs because that local name is not defined in the **caller**.
Reason: A returned value and the function’s internal name are different things.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

My picture explanation, deciding evidence and one limit

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

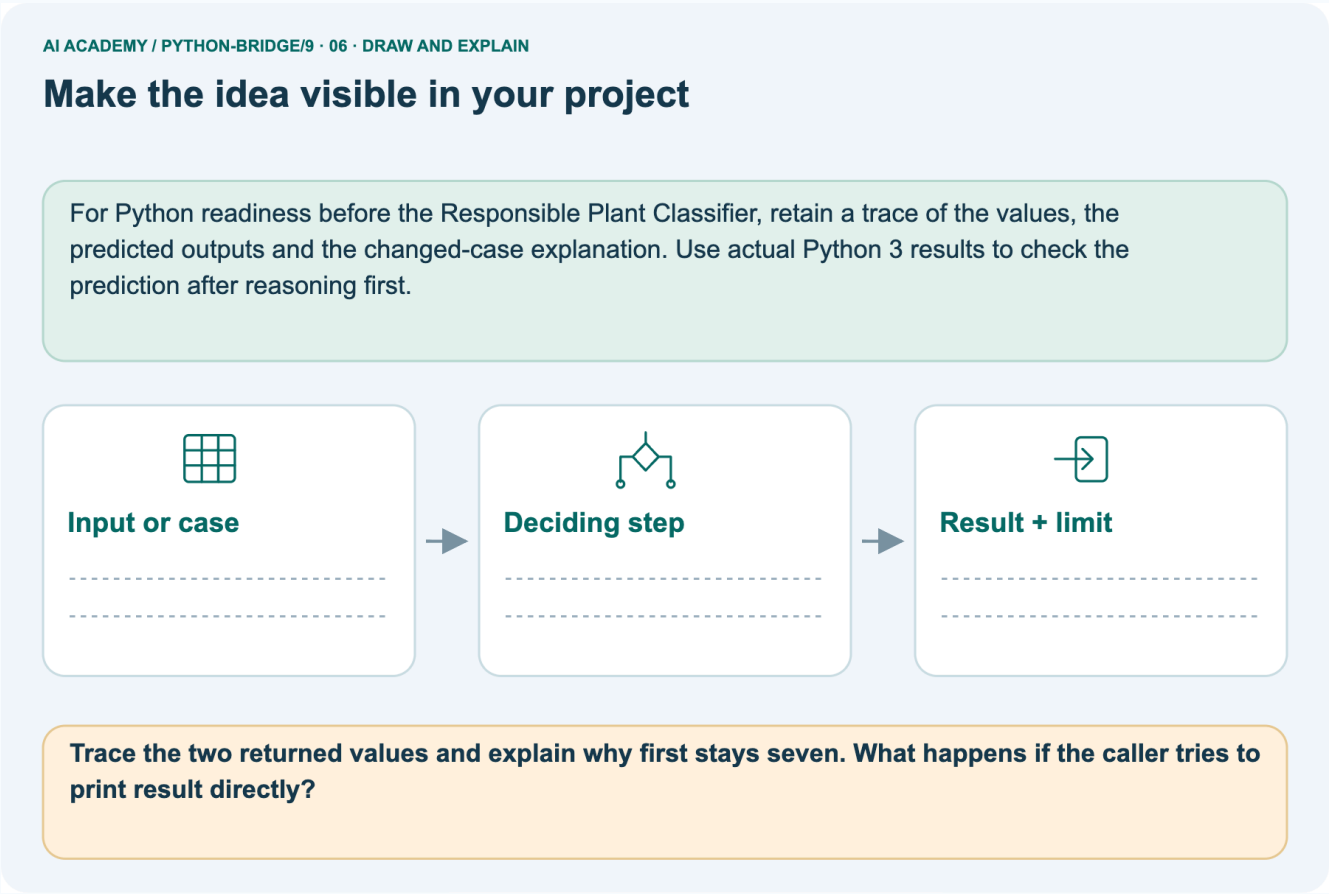
Returning passes a value back to the **caller**. Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Limit: The example uses immutable integers; shared mutable objects require additional reasoning about references.

Explain the case, then point to a limit on the claim.

My picture explanation, deciding evidence and one limit

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

Draw your project trace and justify the deciding step using **evidence**.

My picture explanation, deciding evidence and one limit

▼ Earlier wording, alternatives and picture transcripts

01. Return values and local state: the case

02. Follow the deciding steps

03. Two calls return two values

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.